

Network Traffic Analysis using Wireshark

1. Objective

The objective of this project is to analyze website network traffic using Wireshark and understand how data packets are transmitted between a client and a web server. This project focuses on observing network protocols such as DNS, TCP, HTTP/HTTPS, and TLS, along with studying packet flow, connection establishment, and encrypted communication during website access.

2. Tools Used

- Wireshark
Used for capturing and analyzing network packets.
- Google Chrome
Used to access websites and generate network traffic.
- Windows 10
Operating system used for performing the analysis.

3. Steps Followed

1. Installed and opened Wireshark on the system.
2. Selected the active network interface (Wi-Fi/Ethernet) for packet capturing.
3. Started packet capture in Wireshark.
4. Opened websites using Google Chrome to generate network traffic.
5. Browsed websites for a few minutes to generate sufficient packet data.
6. Stopped the packet capture after collecting the required traffic.
7. Applied filters such as:
 - dns
 - tcp
 - tls
 - http
8. Analyzed different packets and observed:
 - DNS requests and responses
 - TCP 3-way handshake
 - TLS encrypted communication
 - General packet flow between client and server

9. Used the Protocol Hierarchy feature from:

Statistics → Protocol Hierarchy

to observe the percentage and distribution of protocols in the captured traffic.

10. Used the I/O Graph feature from:

Statistics → I/O Graph

to analyze network traffic flow and packet activity over time.

11. Captured screenshots of important packet analysis results, protocol statistics, and traffic graphs.

12. Documented the observations and conclusions obtained from the traffic analysis

4. Protocols Observed

1. DNS (Domain Name System)

Used to translate domain names into IP addresses before establishing communication with websites.

2. TCP (Transmission Control Protocol)

Used for reliable communication between the client and server through connection establishment and packet transmission.

3. TLS (Transport Layer Security)

Used to provide encrypted and secure communication during website access over HTTPS.

4. HTTP/HTTPS

Used for communication between the web browser and web server while accessing websites and web resources.

5. ARP (Address Resolution Protocol)

Used to identify the MAC address associated with an IP address within the local network.

5. Packet Analysis

1. DNS Analysis

No.	Time	Delta Time	Source	Destination	Protocol	Length	Info
37	6.038939100		192.168.1.10	192.168.1.1	DNS	80	Standard query 0xdb34 HTTPS beacons.gcp.gvt2.com
38	6.039312300	0.000373200	192.168.1.10	192.168.1.1	DNS	80	Standard query 0x991c A beacons.gcp.gvt2.com
44	6.055382100	0.016069800	192.168.1.1	192.168.1.10	DNS	167	Standard query response 0xdb34 HTTPS beacons.gcp.gvt2.com CNAME beacons
45	6.057472800	0.002090700	192.168.1.1	192.168.1.10	DNS	126	Standard query response 0x991c A beacons.gcp.gvt2.com CNAME beacons-han
130	6.146224800	0.088752000	192.168.1.10	192.168.1.1	DNS	74	Standard query 0x38f1 HTTPS lh3.google.com
131	6.146565000	0.000340200	192.168.1.10	192.168.1.1	DNS	74	Standard query 0x8497 A lh3.google.com
133	6.161462700	0.014897700	192.168.1.1	192.168.1.10	DNS	144	Standard query response 0x38f1 HTTPS lh3.google.com CNAME lh2.l.google.
134	6.163296100	0.001833400	192.168.1.1	192.168.1.10	DNS	110	Standard query response 0x8497 A lh3.google.com CNAME lh2.l.google.com
168	6.235602300	0.072306200	192.168.1.10	192.168.1.1	DNS	88	Standard query 0xd08a HTTPS ogads-pa.clients6.google.com
169	6.236050400	0.000448100	192.168.1.10	192.168.1.1	DNS	88	Standard query 0x3eca A ogads-pa.clients6.google.com
175	6.250385600	0.014335200	192.168.1.1	192.168.1.10	DNS	138	Standard query response 0xd08a HTTPS ogads-pa.clients6.google.com SOA n
176	6.252424200	0.002038600	192.168.1.1	192.168.1.10	DNS	104	Standard query response 0x3eca A ogads-pa.clients6.google.com A 142.250
188	6.282145800	0.029721600	192.168.1.10	192.168.1.1	DNS	85	Standard query 0x5dde HTTPS lh3.googleusercontent.com
189	6.282504700	0.000358900	192.168.1.10	192.168.1.1	DNS	85	Standard query 0xc1dc A lh3.googleusercontent.com
191	6.297368100	0.014863400	192.168.1.1	192.168.1.10	DNS	171	Standard query response 0x5dde HTTPS lh3.googleusercontent.com CNAME go
192	6.299275400	0.001907300	192.168.1.1	192.168.1.10	DNS	130	Standard query response 0xc1dc A lh3.googleusercontent.com CNAME google
251	7.238495500	0.931220100	192.168.1.10	192.168.1.1	DNS	75	Standard query 0x2ccc HTTPS play.google.com
252	7.238821900	0.000326400	192.168.1.10	192.168.1.1	DNS	75	Standard query 0x70d0 A play.google.com
253	7.245848900	0.015027000	192.168.1.1	192.168.1.10	DNS	125	Standard query response 0x2ccc HTTPS play.google.com SOA ns1.google.com
254	7.247620800	0.001771900	192.168.1.1	192.168.1.10	DNS	91	Standard query response 0x70d0 A play.google.com A 142.251.222.142
286	7.451023100	0.203402300	192.168.1.10	192.168.1.1	DNS	85	Standard query 0x79e2 HTTPS rbs.wd3.myworkdayjobs.com
287	7.451342000	0.000318900	192.168.1.10	192.168.1.1	DNS	85	Standard query 0xbdd8 A rbs.wd3.myworkdayjobs.com
289	7.468634200	0.017292200	192.168.1.1	192.168.1.10	DNS	101	Standard query response 0xbdd8 A rbs.wd3.myworkdayjobs.com A 37.0.1.38
301	7.568665200	0.100031000	192.168.1.1	192.168.1.10	DNS	151	Standard query response 0x79e2 HTTPS rbs.wd3.myworkdayjobs.com SOA dns1
336	8.502678900	0.934013700	192.168.1.10	192.168.1.1	DNS	86	Standard query 0x8601 HTTPS encrypted-tbn0.gstatic.com

Frame 37: Packet, 80 bytes on wire (640 bits), 80 bytes captured (640 bits) on interface \Device\NPF_{A8A5E032-B7D4-49A9-8822-3A14E67E2320}, id 0
Ethernet II, Src: Intel_c7:a8:9e (bc:6e:e2:c7:a8:9e), Dst: GOIPGlobalSe_8c:76:ac (b8:b7:db:8c:76:ac)
Internet Protocol Version 4, Src: 192.168.1.10, Dst: 192.168.1.1
User Datagram Protocol, Src Port: 64990, Dst Port: 53
Domain Name System (query)

DNS Traffic Summary (What's Happening in the Screenshot)

The screenshot shows DNS communication between the client system and DNS server. The system with IP address 192.168.1.10 is sending DNS requests to 192.168.1.1 (router/DNS server) to resolve domain names into IP addresses.

The filter dns is applied in Wireshark, so only DNS packets are visible.

Here's what is happening:

1. Client sends DNS Query

- Your system (192.168.1.10) requests domain information like:
 - lh3.google.com
 - play.google.com
 - googleusercontent.com

2. DNS Server Responds

- The DNS server (192.168.1.1) replies with the corresponding IP address for those domains.

3. Query and Response Process

- You can see:
 - “Standard query” → request sent by client.
 - “Standard query response” → response from DNS server.

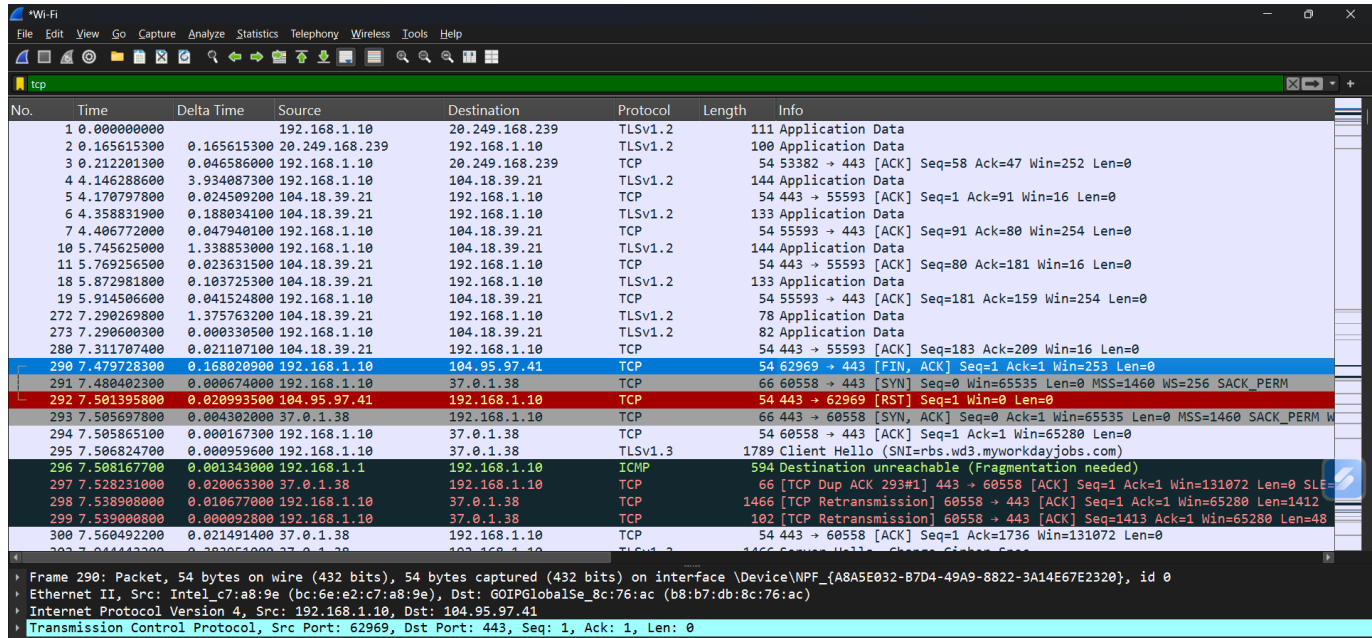
4. Protocol Used

- DNS traffic is happening over UDP Port 53, which is commonly used for faster DNS communication.

5. Purpose

- The system is trying to access Google-related services, and DNS is helping convert domain names into machine-readable IP addresses so the connection can be established.

2. TCP Analysis



No.	Time	Delta Time	Source	Destination	Protocol	Length	Info
1	0.000000000		192.168.1.10	20.249.168.239	TLSv1.2	111	Application Data
2	0.165615300	0.165615300	20.249.168.239	192.168.1.10	TLSv1.2	100	Application Data
3	0.212201300	0.046586000	192.168.1.10	20.249.168.239	TCP	54	53382 → 443 [ACK] Seq=58 Ack=47 Win=252 Len=0
4	4.146288600	3.934087300	192.168.1.10	104.18.39.21	TLSv1.2	144	Application Data
5	4.170797800	0.024509200	104.18.39.21	192.168.1.10	TCP	54	443 → 55593 [ACK] Seq=1 Ack=91 Win=16 Len=0
6	4.358831900	0.188034100	104.18.39.21	192.168.1.10	TLSv1.2	133	Application Data
7	4.406772000	0.047940100	192.168.1.10	104.18.39.21	TCP	54	55593 → 443 [ACK] Seq=91 Ack=80 Win=254 Len=0
10	5.745625000	1.338853000	192.168.1.10	104.18.39.21	TLSv1.2	144	Application Data
11	5.769256500	0.023631500	104.18.39.21	192.168.1.10	TCP	54	443 → 55593 [ACK] Seq=80 Ack=181 Win=16 Len=0
18	5.872981800	0.103725300	104.18.39.21	192.168.1.10	TLSv1.2	133	Application Data
19	5.914506600	0.041524800	192.168.1.10	104.18.39.21	TCP	54	55593 → 443 [ACK] Seq=181 Ack=159 Win=254 Len=0
272	7.290269800	1.375763200	104.18.39.21	192.168.1.10	TLSv1.2	78	Application Data
273	7.290600300	0.000330500	192.168.1.10	104.18.39.21	TLSv1.2	82	Application Data
280	7.311707400	0.021107100	104.18.39.21	192.168.1.10	TCP	54	443 → 55593 [ACK] Seq=183 Ack=209 Win=16 Len=0
290	7.479728300	0.168020900	192.168.1.10	104.95.97.41	TCP	54	62969 → 443 [FIN, ACK] Seq=1 Ack=1 Win=253 Len=0
291	7.480402300	0.000674000	192.168.1.10	37.0.1.38	TCP	66	60558 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
292	7.501395800	0.020993500	104.95.97.41	192.168.1.10	TCP	54	443 → 62969 [RST] Seq=1 Win=0 Len=0
293	7.505697800	0.004302000	37.0.1.38	192.168.1.10	TCP	66	443 → 60558 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 SACK_PERM
294	7.505865100	0.000167300	192.168.1.10	37.0.1.38	TCP	54	60558 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
295	7.506824700	0.000959600	192.168.1.10	37.0.1.38	TLSv1.3	1789	Client Hello (SNI=rbs.wd3.myworkdayjobs.com)
296	7.508167700	0.001343000	192.168.1.1	192.168.1.10	ICMP	594	Destination unreachable (Fragmentation needed)
297	7.528231000	0.020063300	37.0.1.38	192.168.1.10	TCP	66	[TCP Dup ACK 293#1] 443 → 60558 [ACK] Seq=1 Ack=1 Win=131072 Len=0 SLE
298	7.538908000	0.010677000	192.168.1.10	37.0.1.38	TCP	1466	[TCP Retransmission] 60558 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1412
299	7.539008000	0.000092800	192.168.1.10	37.0.1.38	TCP	102	[TCP Retransmission] 60558 → 443 [ACK] Seq=1413 Ack=1 Win=65280 Len=48
300	7.560492200	0.021491400	37.0.1.38	192.168.1.10	TCP	54	443 → 60558 [ACK] Seq=1 Ack=1736 Win=131072 Len=0
301	7.564443300	0.003951100	37.0.1.38	104.18.39.21	TLSv1.2	1466	Server Hello (SNI=rbs.wd3.myworkdayjobs.com)

Frame 290: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF_{A8A5E032-B7D4-49A9-8822-3A14E67E2320}, id 0
Ethernet II, Src: Intel_c7:a8:9e (bc:6e:e2:c7:a8:9e), Dst: GOIPGlobalSe_8c:76:ac (b8:b7:db:8c:76:ac)
Internet Protocol Version 4, Src: 192.168.1.10, Dst: 104.95.97.41
Transmission Control Protocol, Src Port: 62969, Dst Port: 443, Seq: 1, Ack: 1, Len: 0

TCP Traffic Summary (What's Happening in the Screenshot)

The screenshot shows TCP communication between the client system and external servers. The filter tcp is applied in Wireshark, so only TCP packets are visible.

Here's what is happening:

1. Connection Request (TCP Handshake Begins)

- Your system (192.168.1.10) sends a SYN packet to external servers such as 37.0.1.38 using destination port 443 (HTTPS).
- Example: [SYN]
- This indicates the client is trying to establish a connection.

2. Connection Acknowledgment

- The server responds with: [SYN, ACK]
- This means the server accepts the connection request.

3. Connection Established

- The client sends: [ACK]
- Now the TCP three-way handshake is completed and communication starts.

4. Encrypted Data Transfer

- After connection establishment, TLSv1.2/TLSv1.3 Application Data packets are visible.
- This means secure HTTPS communication is happening.
- Example: TLSv1.3 Client Hello
- The client initiates secure encrypted communication.

5. Website/Service Access

- The packet: Client Hello (SNI=rbs.wd3.myworkdayjobs.com)
- indicates the system is trying to securely connect to a Workday jobs website through HTTPS.

6. TCP Errors / Network Issue

- Some packets show: TCP Retransmission
 - meaning packets were resent because acknowledgment was delayed or lost.
- Also: Destination unreachable (Fragmentation needed)
 - indicates a network issue where packet size exceeded allowed transmission limits and fragmentation was required.

7. Connection Termination

- Packet: [FIN, ACK]
- indicates the TCP connection is being closed properly

3.TLS Analysis

No.	Time	Delta Time	Source	Destination	Protocol	Length	Info
260	7.268225300	0.018222900	142.251.222.142	192.168.1.10	QUIC	1296	Initial, SCID=e30bf92fae53eee5, PKN: 3, CRYPTO, PADDING
272	7.290269800	0.022044500	104.18.39.21	192.168.1.10	TLSv1.2	78	Application Data
273	7.290600300	0.000305000	192.168.1.10	104.18.39.21	TLSv1.2	82	Application Data
295	7.506824700	0.216224400	192.168.1.10	37.0.1.38	TLSv1.3	1789	Client Hello (SNI=rbs.wd3.myworkdayjobs.com)
296	7.508167700	0.001343000	192.168.1.1	192.168.1.10	ICMP	594	Destination unreachable (Fragmentation needed)
302	7.944443200	0.436275500	37.0.1.38	192.168.1.10	TLSv1.3	1466	Server Hello, Change Cipher Spec
305	7.944443200	0.000000000	37.0.1.38	192.168.1.10	TLSv1.3	63	Application Data
310	7.950632500	0.006189300	192.168.1.10	37.0.1.38	TLSv1.3	118	Change Cipher Spec, Application Data
312	8.147924200	0.197291700	37.0.1.38	192.168.1.10	TLSv1.3	582	Application Data, Application Data
341	8.521911000	0.373986800	192.168.1.10	142.250.192.206	QUIC	1292	Initial, DCID=aabdba3318c276bd, PKN: 2, PING, CRYPTO, CRYPTO, PADDING,
347	8.570213900	0.048302900	142.250.192.206	192.168.1.10	QUIC	1292	Initial, SCID=eabdba3318c276bd, PKN: 3, CRYPTO, PADDING
348	8.570213900	0.000000000	142.250.192.206	192.168.1.10	QUIC	1292	Initial, SCID=eabdba3318c276bd, PKN: 4, CRYPTO, PADDING
360	8.618929800	0.048715900	192.168.1.10	142.250.192.206	TLSv1.3	1854	Client Hello (SNI=encrypted-tbn0.gstatic.com)
373	8.676037200	0.057107400	142.250.192.206	192.168.1.10	TLSv1.3	1466	Server Hello, Change Cipher Spec
376	8.676206900	0.000169700	142.250.192.206	192.168.1.10	TLSv1.3	1212	Application Data
379	8.676652400	0.000445500	192.168.1.10	142.250.192.206	TLSv1.3	128	Change Cipher Spec, Application Data
380	8.732348500	0.055696100	142.250.192.206	192.168.1.10	TLSv1.3	1036	Application Data, Application Data
401	10.263433900	1.531085400	192.168.1.10	142.250.193.78	QUIC	1292	Initial, DCID=85ad75ecacd55307, PKN: 2, PING, PADDING, CRYPTO, CRYPTO,
407	10.318227600	0.054793700	142.250.193.78	192.168.1.10	QUIC	1292	Initial, SCID=e5ad75ecacd55307, PKN: 3, CRYPTO, PADDING
408	10.318519000	0.000291400	142.250.193.78	192.168.1.10	QUIC	1292	Initial, SCID=e5ad75ecacd55307, PKN: 4, CRYPTO, PADDING
418	10.354507600	0.035988600	192.168.1.10	142.250.193.78	TLSv1.3	1874	Client Hello (SNI=ogs.google.com)
429	10.405416700	0.050909100	142.250.193.78	192.168.1.10	TLSv1.3	1466	Server Hello, Change Cipher Spec
435	10.406273900	0.000857200	142.250.193.78	192.168.1.10	TLSv1.3	873	Application Data
438	10.407718200	0.001436300	192.168.1.10	142.250.193.78	TLSv1.3	128	Change Cipher Spec, Application Data
454	10.459737000	0.052026800	142.250.193.78	192.168.1.10	TLSv1.3	1012	Application Data, Application Data
465	10.607000000	0.147263000	192.168.1.10	172.217.26.00	QUIC	1003	Initial, DCID=150c706c74a7b7, PKN: 2, PING, PING, CRYPTO, CRYPTO, CRY

Frame 273: Packet, 82 bytes on wire (656 bits), 82 bytes captured (656 bits) on interface \Device\NPF_{A8A5E032-B7D4-49A9-8822-3A14E67E2320}, id 0
 Ethernet II, Src: Intel_c7:a8:9e (bc:6e:e2:c7:a8:9e), Dst: GOIPGlobalSe_8c:76:ac (b8:b7:db:8c:76:ac)
 Internet Protocol Version 4, Src: 192.168.1.10, Dst: 104.18.39.21
 Transmission Control Protocol, Src Port: 55593, Dst Port: 443, Seq: 181, Ack: 183, Len: 28
 Transport Layer Security

TLS Traffic Summary (What's Happening in the Screenshot)

The screenshot shows TLS (Transport Layer Security) communication between the client system and external servers. The filter `tls` is applied in Wireshark, so encrypted secure traffic is visible.

Here's what is happening:

1. Secure Connection Initiation

- The client system (192.168.1.10) starts a secure connection with external servers over port 443 (HTTPS).
- TLS is used to encrypt communication and protect data from attackers.

2. TLS Handshake Begins

- The packet: **Client Hello**
 - means the client is requesting to start a secure encrypted session with the server.

Example from screenshot: **Client Hello (SNI=rbs.wd3.myworkdayjobs.com)**

- This indicates the client is attempting to connect securely to a website/service.

3. Server Response

- The server replies with: **Server Hello**
 - meaning the server accepts the secure connection and negotiates encryption settings.

4. Cipher Exchange

- Packet: **Change Cipher Spec**
 - shows that both client and server are switching to encrypted communication.

5. Encrypted Data Transfer

- After the handshake, packets show: **Application Data**
 - meaning actual encrypted data is being transferred securely.

6. Multiple Secure Connections

- The screenshot shows connections to domains like:
 - `rbs.wd3.myworkdayjobs.com`
 - `encrypted-tbn0.gstatic.com`
 - `ogs.google.com`
- indicating secure communication with different web services.

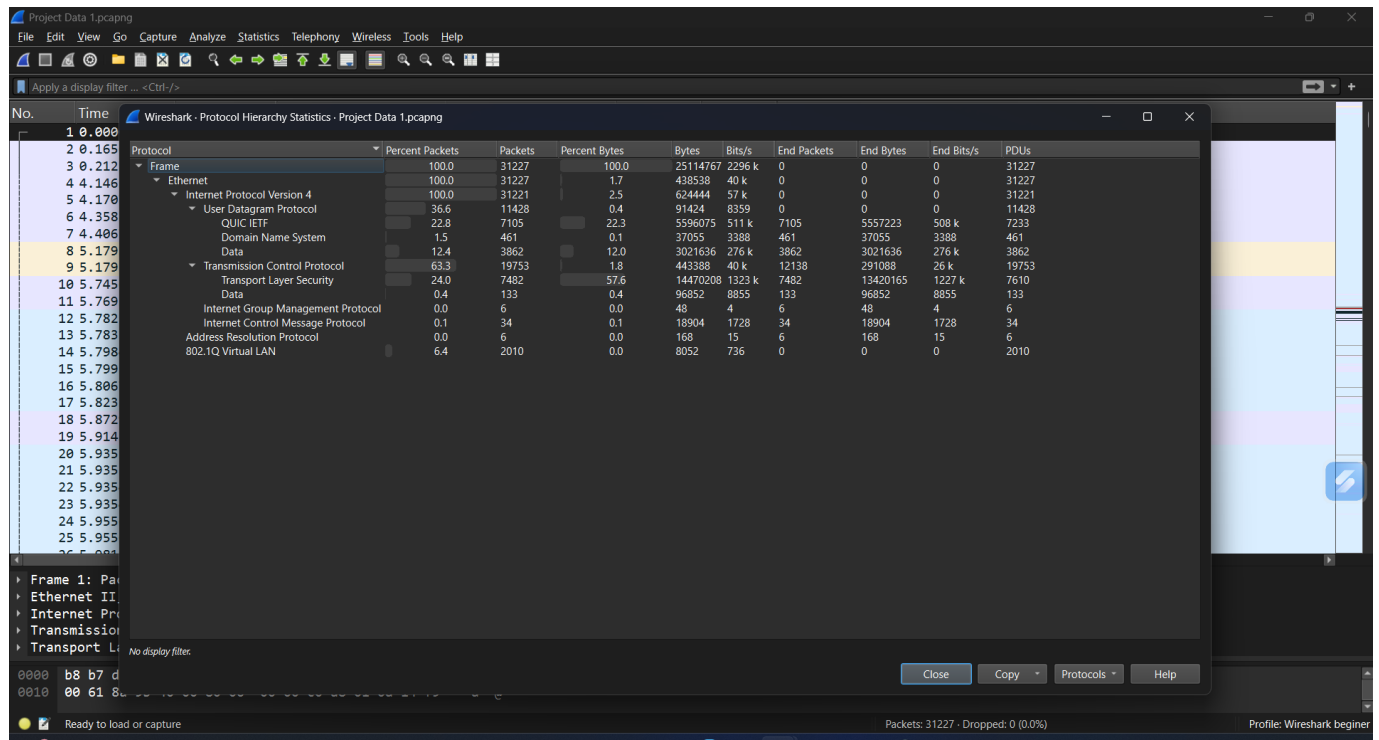
7. QUIC Protocol

- Some packets show: **QUIC**
 - QUIC is a modern secure transport protocol used by browsers for faster encrypted communication, especially in Google services.

4.HTTP Analysis

- No significant HTTP packets were observed because most modern websites use HTTPS with TLS encryption for secure communication.

6. Protocol Hierarchy Statistics Summary



Protocol Hierarchy Statistics Summary (What's Happening in the Screenshot)

The screenshot shows **Protocol Hierarchy Statistics in Wireshark**, which provides an overview of all network protocols used during packet capture and their traffic percentage.

Here's what is happening:

1. Total Captured Traffic

- Total packets captured: **31,227**
- The capture includes different protocols involved in network communication.

2. IPv4 Communication

- Most traffic is under: **Internet Protocol Version 4 (IPv4)**
- meaning the network communication mainly uses IPv4 addressing.

3. TCP Traffic Dominance

- **TCP (Transmission Control Protocol)** contributes around **63.3% of packets**.
- This indicates most communication is **connection-oriented and reliable**, mainly web browsing and secure communication.

4. TLS Encrypted Traffic

- **TLS (Transport Layer Security)** contributes around **24% of packets** and a higher percentage of bytes.
- This shows a large amount of communication is **encrypted HTTPS traffic**, ensuring secure internet access.

5. UDP Traffic

- **UDP (User Datagram Protocol)** contributes around **36.6% of packets**.
- UDP is used for faster communication where speed matters more than reliability.

6. QUIC Protocol Usage

- **QUIC IETF** contributes around **22.8% of packets**.
- QUIC is used for **fast and secure communication**, mainly by modern browsers and Google services.

7. DNS Traffic

- **Domain Name System (DNS)** traffic is present.
- DNS resolves domain names into IP addresses before connections are established.

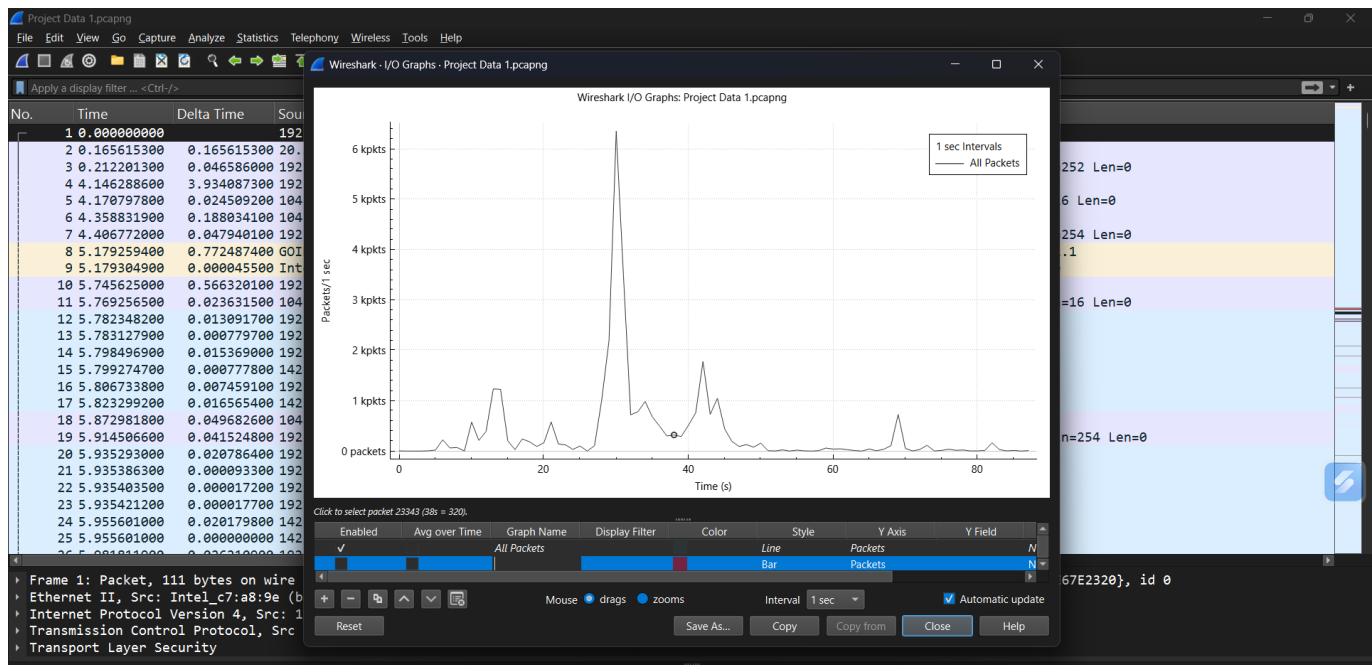
8. Other Supporting Protocols

- **ICMP** → used for network diagnostics and error reporting.
- **ARP (Address Resolution Protocol)** → maps IP addresses to MAC addresses in local networks.
- **802.1Q VLAN** → indicates virtual LAN tagging in network traffic.

Note:

The network traffic mainly consists of **TCP and TLS encrypted communication**, indicating secure web browsing activity. QUIC and DNS protocols support faster communication and domain resolution.

7. I/O Graph



I/O Graph Summary (What's Happening in the Screenshot)

The screenshot shows the **Wireshark I/O Graph**, which represents the **network traffic flow over time** during packet capture.

Here's what is happening:

1. Traffic Monitoring Over Time

- The graph displays **network packets per second** during the capture session.
- **X-axis (Horizontal)** → Time in seconds.
- **Y-axis (Vertical)** → Number of packets transferred per second.

2. Normal Network Activity

- At the beginning, traffic remains **low and stable**, indicating normal communication activity.

3. Traffic Spike Observed

- Around **30–35 seconds**, there is a **major spike in packet transmission**, reaching nearly **6k packets/sec**.
- This indicates a sudden increase in network activity, possibly due to:
 - Website loading
 - Multiple requests/responses
 - Background application communication
 - Secure web traffic (TLS/HTTPS)

4. Moderate Traffic Bursts

- Around **40–45 seconds**, smaller spikes are visible.
- These represent temporary increases in communication between the client and external servers.

5. Low Traffic Period

- After the peak, traffic decreases and becomes relatively stable, indicating reduced network activity.

6. Packet Flow Observation

- The graph shows that traffic is **not constant**, but varies depending on user activity and background network communication.

Note:

The captured traffic experienced a **high network activity peak around 30–35 seconds**, followed by smaller bursts and stable communication, showing dynamic network behavior during internet usage.

8. Findings and Observations

- DNS packets were observed during domain name resolution before establishing website communication.
- TCP protocol was used to establish reliable communication between the client and server through the 3-way handshake process.
- TLS packets indicated encrypted and secure communication during HTTPS website access.
- Multiple packets were exchanged continuously while loading and browsing websites.
- Protocol Hierarchy analysis showed that TCP and TLS occupied the majority of captured network traffic.
- I/O Graph analysis displayed traffic spikes during webpage loading and user interaction activities.
- No HTTP packets were observed because modern websites mainly use HTTPS encryption.

9. Conclusion

This project successfully analyzed website traffic using Wireshark. Different network protocols such as DNS, TCP, TLS, and HTTP/HTTPS were observed and studied during packet analysis. The project helped in understanding packet flow, domain name resolution, connection establishment, and encrypted communication in modern web traffic. Additionally, features like Protocol Hierarchy and I/O Graphs provided better insight into overall network activity and protocol distribution.

